

透過 Google Kubernetes Engine (GKE) 部署、擴充及更新網站

來源：<https://codelabs.developers.google.com/codelabs/cloud-deploy-website-on-gke?hl=zh-tw>

步驟數：12

瞭解如何透過 GKE 部署、擴充及更新網站。

目錄

1. [簡介](#)
2. [環境設定](#)
3. [建立 GKE 叢集](#)
4. [複製來源存放區](#)
5. [使用 Cloud Build 建立 Docker 容器](#)
6. [將容器部署至 GKE](#)
7. [公開 GKE Deployment](#)
8. [擴充 GKE Deployment](#)
9. [變更網站內容](#)
10. [更新網站，完全不必停機](#)
11. [清除所用資源](#)
12. [恭喜！](#)

步驟 1 · 約 5 分鐘

1. 簡介

經營網站和應用程式並不容易。

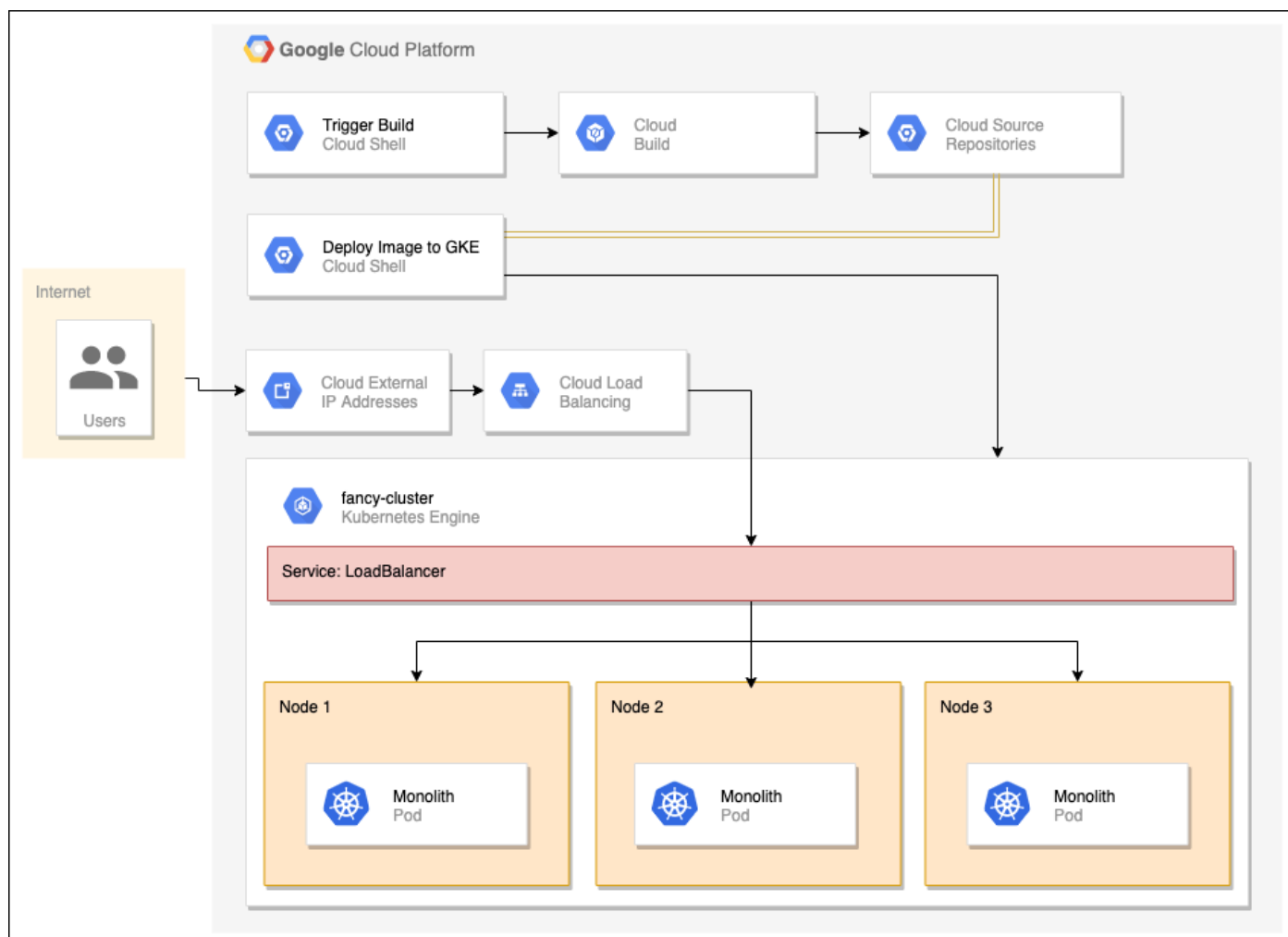
當不該出錯的地方出錯、伺服器當機、需求增加導致使用更多資源，以及在不中斷服務的情況下進行變更，都會使作業變得複雜，令人備感壓力。

想像一下，如果有一項工具可以協助您完成上述所有作業，甚至還能自動執行，會怎麼樣？有了 GKE，您不僅可以輕鬆完成上述所有作業，在本程式碼研究室中，您將扮演虛構公司 Fancy Store 的開發人員，負責管理電子商務網站。由於出現擴充和中斷問題，您必須將應用程式部署到 GKE！

本實驗室包含多項練習，順序與常見雲端開發作業類似，包括：

1. 建立 GKE 叢集。
2. 建立 Docker 容器。
3. 將容器部署至 GKE。
4. 透過 Service 公開容器。
5. 將容器擴充至多個副本。
6. 修改網站。
7. 在不停機的狀態下推出新版本。

架構圖



課程內容

- 如何建立 GKE 叢集
- 如何建立 Docker 映像檔
- 如何將 Docker 映像檔部署至 Kubernetes
- 如何在 Kubernetes 上調度應用程式資源
- 如何在 Kubernetes 上執行滾動式更新

必要條件

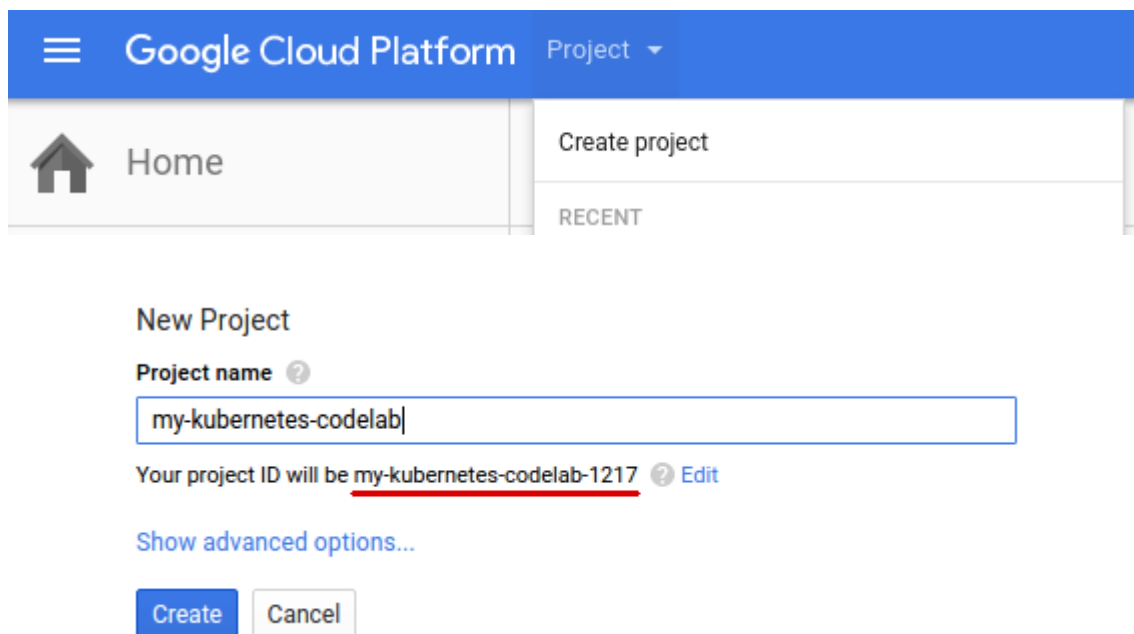
- 具備管理員存取權的 Google 帳戶，可建立專案；或具備專案擁有者角色的專案
 - Docker 和 Kubernetes 的基本概念 (如果缺乏基本概念，請立即複習 [Docker](#) 和 [Kubernetes](#))
-

步驟 2 · 約 5 分鐘

2. 環境設定

自修實驗室環境設定

如果沒有 Google 帳戶，請先[建立帳戶](#)。登入 [Google Cloud 控制台](#)，然後建立新專案。



請注意，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。這個名稱稍後會稱為 `PROJECT_ID`。

接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。Google Cloud 新使用者享有價值 **\$300 美元的免費試用期**。如果您不是新使用者，請放心，因為程式碼研究室的費用應該不會超過幾美元。不過，如果您使用更多資源或讓資源繼續執行，程式碼研究室可能會產生更多費用 (請參閱結尾的「清除」一節)。詳情請參閱「[定價](#)」。

Cloud Shell

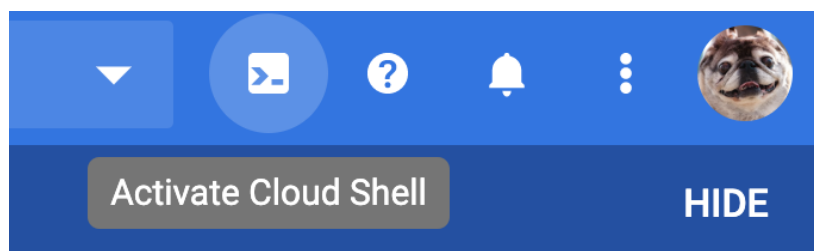
您可以使用筆電遠端操作 Google Cloud 和 GKE，但本程式碼研究室會使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

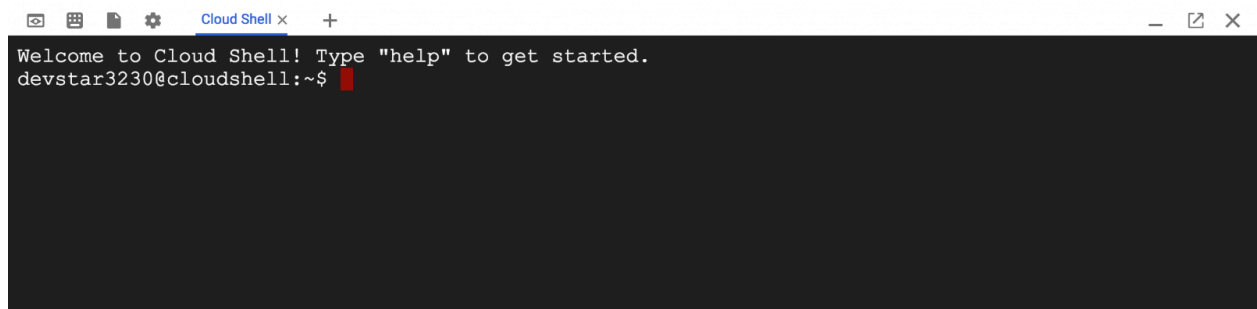
這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



即可 (佈建並連線至環境的作業需要一些時間才能完成)。





連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

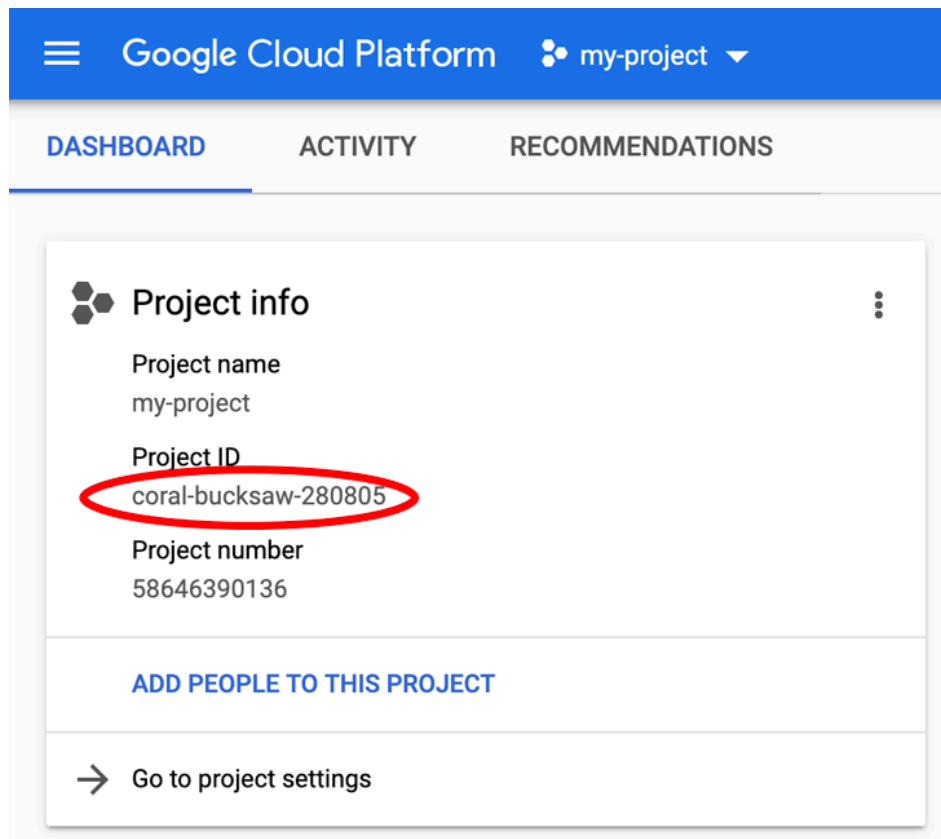
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

3. 建立 GKE 叢集

現在您已建立可運作的開發環境，接下來需要 GKE 叢集，才能部署網站。建立叢集前，請確認已啟用適當的 API。執行下列指令來啟用 Containers API：

```
gcloud services enable container.googleapis.com
```

現在可以建立叢集了！請按照下列步驟建立名為 **fancy-cluster** 的叢集，其中包含 3 個節點：

```
gcloud container clusters create fancy-cluster --num-nodes 3
```

警告

如果收到未指定區域/可用區的錯誤訊息，請參閱環境設定一節，確認已設定預設運算可用區。

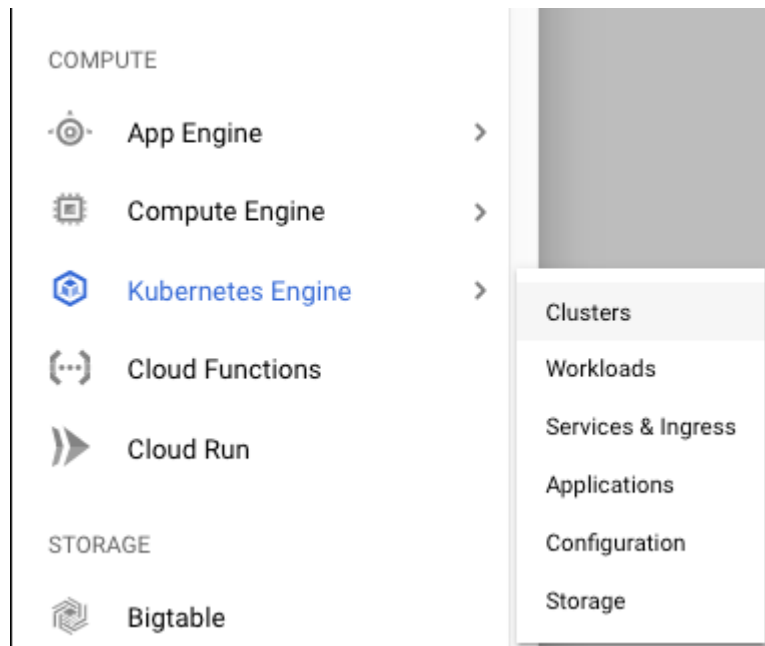
可能需要幾分鐘時間才能建立叢集。接著執行下列指令，查看叢集的三個工作站虛擬機器 (VM) 執行個體：

```
gcloud compute instances list
```

輸出內容：

NAME	ZONE	MACHINE_TYPE	PREEMPTIBLE
INTERNAL_IP	EXTERNAL_IP	STATUS	
gke-fancy-cluster-default-pool-ad92506d-1ng3	us-east4-a	n1-standard-1	
10.150.0.7	XX.XX.XX.XX	RUNNING	
gke-fancy-cluster-default-pool-ad92506d-4fvq	us-east4-a	n1-standard-1	
10.150.0.5	XX.XX.XX.XX	RUNNING	
gke-fancy-cluster-default-pool-ad92506d-4zs3	us-east4-a	n1-standard-1	
10.150.0.6	XX.XX.XX.XX	RUNNING	

您也可以在 Cloud 控制台中查看叢集和相關資訊。按一下左上角的選單按鈕，向下捲動至 Kubernetes Engine，然後按一下「叢集」。您應該會看到名為 **fancy-cluster** 的叢集。



Kubernetes clusters + CREATE CLUSTER + DEPLOY ↻ REFRESH 🗑 DELETE SHOW INFO PANEL

A Kubernetes cluster is a managed group of VM instances for running containerized applications. [Learn more](#)

Filter by label or name

☐	Name ^	Location	Cluster size	Total cores	Total memory	Notifications	Labels
☐	✅ fancy-cluster	us-east4-a	3	3 vCPUs	11.25 GB	<button>Connect</button>	

恭喜！您已建立第一個叢集！

注意事項

如果您使用現有的 GKE 叢集，或是已透過 Cloud Console 建立叢集，則需要執行以下指令來擷取叢集的憑證，並設定 kubectl 指令列工具：

```
gcloud container clusters get-credentials fancy-cluster
```

如果您已使用上述的 gcloud container clusters create 指令建立叢集，則無需完成此步驟。

步驟 4 · 約 5 分鐘

4. 複製來源存放區

這是現成的網站，您只需要從存放區複製原始碼，就能專心建立 Docker 映像檔並部署至 GKE。

執行下列指令，將來源存放區複製到 Cloud Shell 執行個體，並切換至適當的目錄。您也需要安裝 Node.js 依附元件，以便在部署前測試應用程式。

```
cd ~  
git clone https://github.com/googlecodelabs/monolith-to-microservices.git  
cd ~/monolith-to-microservices  
./setup.sh
```

這會複製存放區、變更目錄，並安裝在本機執行應用程式所需的依附元件。這項指令碼可能需要幾分鐘才能執行完畢。

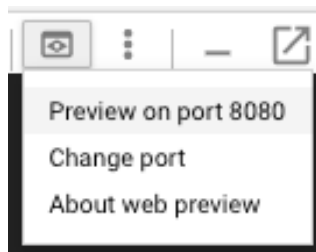
請審慎評估並測試應用程式。執行下列指令啟動網路伺服器：

```
cd ~/monolith-to-microservices/monolith  
npm start
```

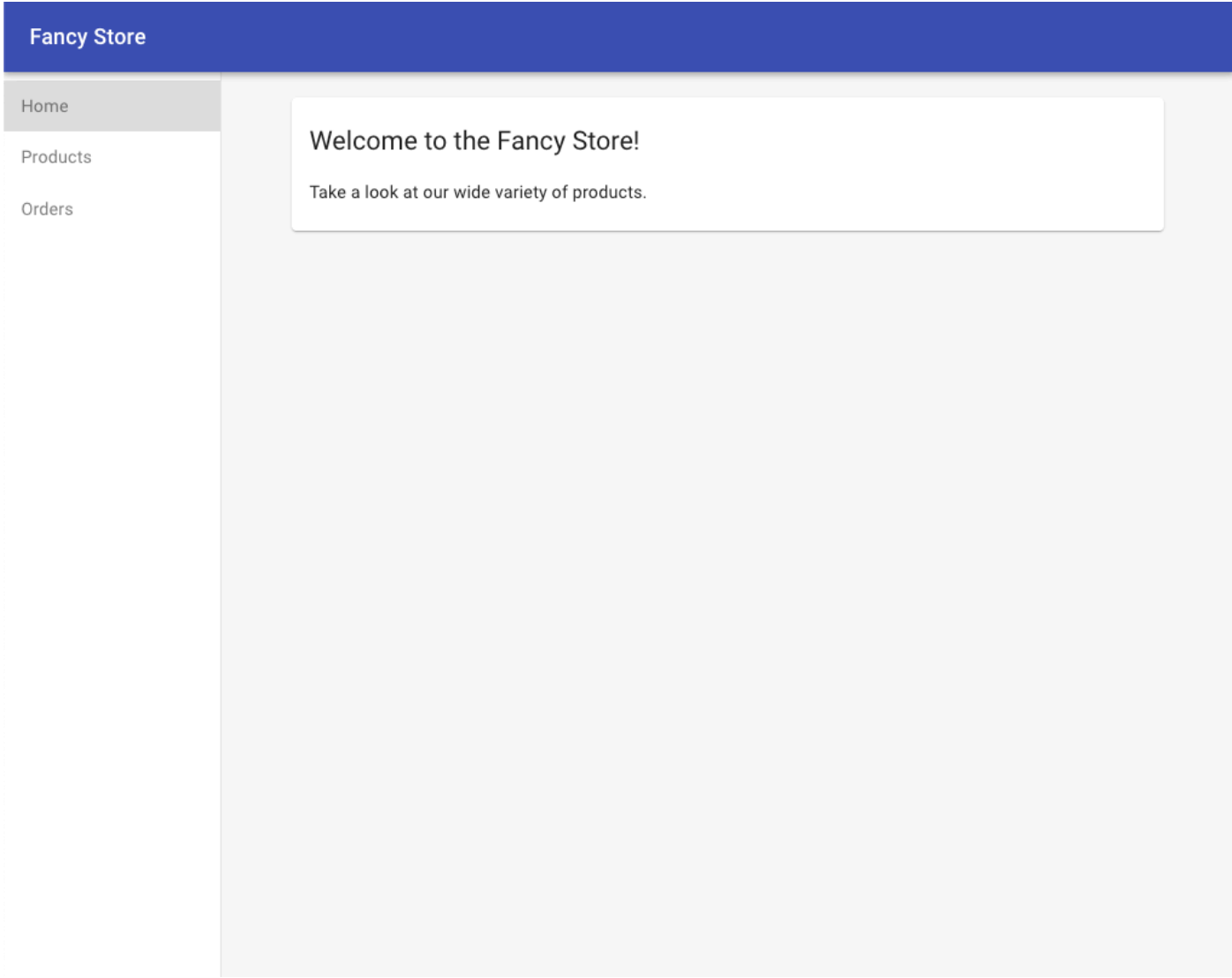
輸出內容：

```
Monolith listening on port 8080!
```

點選 Cloud Shell 選單中的「網頁預覽」圖示，然後選取「透過以下通訊埠預覽：8080」，即可預覽應用程式。



系統應會開啟新視窗，顯示 Fancy Store 的實際運作情形！



查看網站後，即可關閉該視窗。在終端機視窗中按下 `Control+C` 鍵 (Windows 或 Mac)，停止網路伺服器程序。

5. 使用 Cloud Build 建立 Docker 容器

準備好原始碼檔案後，就可以開始將應用程式 Docker 化了！

一般來說，您必須採取兩個步驟：建構 Docker 容器，然後將容器推送至存放區，以便儲存映像檔供 GKE 提取。不過，只要一個指令，Cloud Build 就能建構 Docker 容器，並將映像檔放入 Container Registry，讓您輕鬆完成這項工作！（如要查看手動建立 Docker 檔案並推送的程序，請參閱 [Container Registry 快速入門導覽課程](#)）。

Cloud Build 會壓縮目錄中的檔案，並移至 Cloud Storage bucket。建構程序會從 bucket 擷取檔案，並使用 Dockerfile 執行 Docker 建構程序。您在 Docker 映像檔中指定了 `--tag` 旗標，並將主機設為 `gcr.io`，產生的 Docker 映像檔會推送至 Container Registry。

首先，請執行下列指令來啟用 Cloud Build API：

```
gcloud services enable cloudbuild.googleapis.com
```

啟用 API 後，請在 Cloud Shell 執行下列指令來啟動建構程序：

```
cd ~/monolith-to-microservices/monolith
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:1.0.0 .
```

這個程序需要幾分鐘，完成後終端機中會顯示類似以下的輸出內容：

```
-----
-----
-----
ID                                CREATE_TIME                                DURATION  SOURCE
IMAGES                            STATUS
1ae295d9-63cb-482c-959b-bc52e9644d53  2019-08-29T01:56:35+00:00  33S
gs://<PROJECT_ID>_cloudbuild/source/1567043793.94-
abfd382011724422bf49af1558b894aa.tgz  gcr.io/<PROJECT_ID>/monolith:1.0.0  SUCCESS
```

如要查看建構歷程記錄或即時監看程序，請前往 Cloud Console。按一下左上角的選單按鈕，向下捲動至「Ci/CD」，然後依序點選「Cloud Build」和「記錄」。您會看到先前建構作業的清單，但應該只有您建立的建構作業。

← Build History

Build	Source	Git commit	Trigger	Started	Image target	Image tag
✓ 985ce3e5-afba...	gs://gcb-docs-project_cloudbuild/source/1508159187.8-b0d8841d51674a30aebd1e55bb99486f.tgz	—	—	9:06 AM	gcr.io/gcb-docs-project/quickstart-image	latest
✓ d2e89f1b-cbf9...	gs://gcb-docs-project_cloudbuild/source/1508158566.55-725755714baa4b7e9e99984c422ec4e2.tgz	—	—	8:56 AM	gcr.io/gcb-docs-project/quickstart-image	latest

點選「建構作業 ID」，即可查看該建構作業的所有詳細資料，包括記錄輸出內容。

在「建構作業詳細資料」頁面，點選「建構資訊」部分中的「映像檔名稱」，即可查看建立的容器映像檔。

Status	✔ Build successful
Build id	fbcd08-44bc-46e8-bd18-ba3465ade0cb
Image	gcr.io/gcb-docs-project/quickstart-image
Source	gs://gcb-docs-project_cloudbuild/source/1534173107.74-8167ff55ae2e4b8ca34779daf3090eed.tgz
Started	August 13, 2018 at 11:11:49 AM UTC-4
Build time	15 sec

Build steps

✔ gcr.io/cloud-builders/docker	0.7 sec ▾
build -t gcr.io/gcb-docs-project/quickstart-image .	

6. 將容器部署至 GKE

您已將網站容器化，並將容器推送至 Container Registry，現在可以部署至 Kubernetes 了！

如要在 GKE 叢集上部署和管理應用程式，您必須與 Kubernetes 叢集管理系統進行通訊。一般來說，此操作會透過 **kubectl** 指令列工具進行。

Kubernetes 會以 **Pod** 的形式呈現應用程式；Pod 是容器/緊耦合容器群組的單位，也是 Kubernetes 中最小的可部署單位。在這裡，每個 Pod 只包含單體容器。

如要部署應用程式，請建立 **Deployment**。Deployment 會管理應用程式的多個副本（稱為副本），並將這些副本安排在叢集各個節點上執行。就這個範例而言，部署作業只會執行應用程式的一個 Pod。Deployment 會建立 **ReplicaSet**，確保 Pod 數量符合預期。ReplicaSet 負責確保指定數量的副本隨時都在執行。

`kubectl create deployment` 指令會讓 Kubernetes 在叢集上建立名為 **monolith** 的 Deployment，並包含 **1** 個副本。

執行下列指令來部署應用程式：

```
kubectl create deployment monolith --
image=gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:1.0.0
```

注意事項

最佳做法是使用 YAML 檔案和 GitHub 或 Cloud Source Repositories 等原始碼控管系統來儲存這些變更。詳情請參閱「[部署作業](#)」。

驗證部署

執行下列指令，確認 Deployment 已順利建立 (Pod 狀態可能需要幾分鐘才會變成「Running」)：

```
kubectl get all
```

輸出內容：

NAME	READY	STATUS	RESTARTS	AGE
pod/monolith-7d8bc7bf68-htm7z	1/1	Running	0	6m21s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.27.240.1	<none>	443/TCP	24h

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/monolith	1	1	1	1	20m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/monolith-7d8bc7bf68	1	1	1	20m

輸出內容會顯示幾項資訊：您會看到目前的 Deployment、ReplicaSet (所需 Pod 數量為 1) 和正在執行的 Pod。看來一切都順利建立完成！

實用提示

您也可以透過 Cloud 控制台查看 Kubernetes 部署項目。前往左上方的選單，然後依序點選「Kubernetes Engine」>「工作負載」。

注意事項

如果看到非預期的錯誤或狀態，可以使用下列指令對資源偵錯，查看詳細資訊：

```
kubectl describe pod monolith
```

```
kubectl describe pod/monolith-7d8bc7bf68-2bxts
```

```
kubectl describe deployment monolith
```

```
kubectl describe deployment.apps/monolith
```

在輸出內容的結尾可以看到錯誤事件清單和資源的詳細資訊。

如要個別查看資源，可以執行下列指令：

```
# Show pods
kubectl get pods

# Show deployments
kubectl get deployments

# Show replica sets
kubectl get rs

#You can also combine them
kubectl get pods,deployments
```

如要充分瞭解 Kubernetes 的優點，請模擬伺服器當機，刪除 Pod，看看會發生什麼事！

複製前一個指令的 Pod 名稱，然後執行下列指令來刪除 Pod：

```
kubectl delete pod/<POD_NAME>
```

如果動作夠快，可以再次執行上述指令，應該會看到兩個 Pod：一個正在終止，另一個正在建立或執行：

```
kubectl get all
```

輸出內容：

NAME	READY	STATUS	RESTARTS	AGE
pod/monolith-7d8bc7bf68-2bxts	1/1	Running	0	4s
pod/monolith-7d8bc7bf68-htm7z	1/1	Terminating	0	9m35s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.27.240.1	<none>	443/TCP	24h

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/monolith	1	1	1	1	24m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/monolith-7d8bc7bf68	1	1	1	24m

Why did that happen? ReplicaSet 發現 Pod 正在終止，因此觸發建立新的 Pod，以維持所需的副本數量。稍後您將瞭解如何擴充規模，確保有多個執行個體運作，這樣即使其中一個執行個體故障，使用者也不會遇到停機時間！

7. 公開 GKE Deployment

您已將應用程式部署至 GKE，但無法從叢集外部存取。根據預設，您在 GKE 上執行的容器無法從網際網路存取，因為這些容器沒有外部 IP 位址。您必須透過 [Service](#) 資源，明確將應用程式公開至網際網路傳出的流量。Service 可為應用程式的 Pod 提供網路和 IP 支援。GKE 會為應用程式建立外部 IP 和負載平衡器 ([需要計費](#))。

執行下列指令，將網站公開至網際網路：

```
kubectl expose deployment monolith --type=LoadBalancer --port 80 --target-port 8080
```

輸出內容：

```
service/monolith exposed
```

存取服務

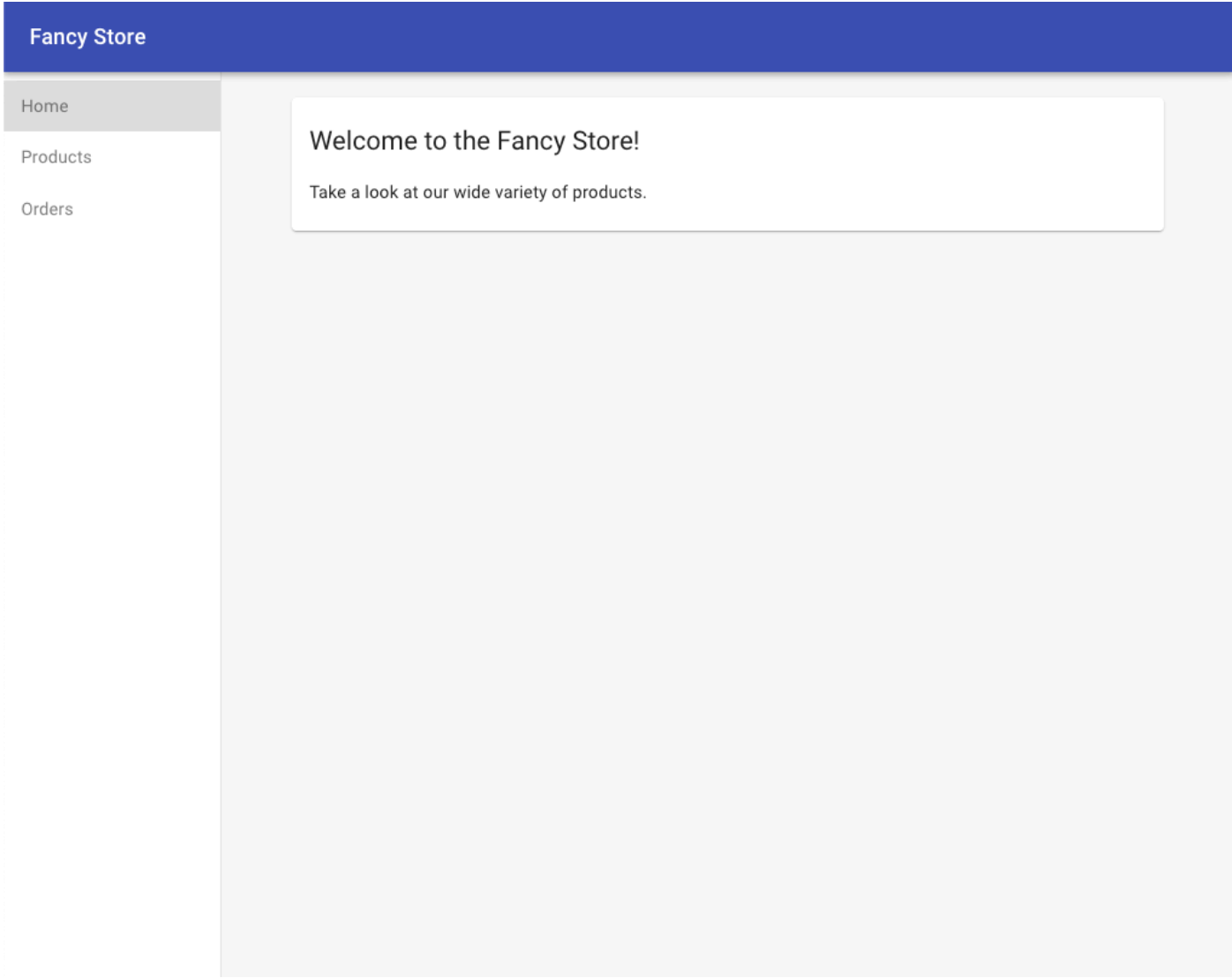
GKE 會將外部 IP 位址指派給 Service 資源，而非 Deployment。如要找出 GKE 為應用程式佈建的外部 IP，請透過 `kubectl get service` 指令檢查該 Service：

```
kubectl get service
```

輸出內容：

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
monolith	10.3.251.122	203.0.113.0	80:30877/TCP	3d

確認應用程式的外部 IP 位址後，請複製該位址。將瀏覽器指向該網址 (如 `http://203.0.113.0`)，檢查應用程式是否能存取。



您應該會看到先前測試的網站。恭喜！您的網站已完全在 Kubernetes 上運作！

步驟 8 · 約 5 分鐘

8. 擴充 GKE Deployment

現在您的應用程式已在 GKE 中執行，並公開發布到網際網路。假設您的網站變得非常熱門，您需要將應用程式擴充至多個執行個體，才能處理流量。瞭解如何將應用程式擴充至最多三個副本。

執行下列指令，將部署作業擴充至三個副本：

```
kubectl scale deployment monolith --replicas=3
```

輸出內容：

```
deployment.apps/monolith scaled
```

驗證擴充的部署作業

執行下列指令，確認 Deployment 已順利擴充：

```
kubectl get all
```

輸出內容：

NAME	READY	STATUS	RESTARTS	AGE
pod/monolith-7d8bc7bf68-2bxts	1/1	Running	0	36m
pod/monolith-7d8bc7bf68-7ds7q	1/1	Running	0	45s
pod/monolith-7d8bc7bf68-c5kxk	1/1	Running	0	45s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.27.240.1	<none>	443/TCP	25h
service/monolith	LoadBalancer	10.27.253.64	XX.XX.XX.XX	80:32050/TCP	6m7s

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/monolith	3	3	3	3	61m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/monolith-7d8bc7bf68	3	3	3	61m

現在應該會看到 3 個 Pod 執行個體。另請注意，Deployment 和 ReplicaSet 現在的所需數量都是 3。

步驟 9 · 約 0 分鐘

9. 變更網站內容

行銷團隊要求您變更網站首頁。希望包含更多公司簡介與銷售產品等資訊。在本節中，您要在首頁加入一些文字，滿足行銷團隊的要求！看來有某位開發人員已做了變更，檔案名稱為 `index.js.new`。您可以將檔案複製到 `index.js`，網站應該就會反映新內容。按照下方說明操作，進行適當變更。

執行下列指令，將更新後的檔案複製為正確檔名，並顯示檔案內容來確認變更：

```
cd ~/monolith-to-microservices/react-app/src/pages/Home
mv index.js.new index.js
cat ~/monolith-to-microservices/react-app/src/pages/Home/index.js
```

顯示的程式碼應如下所示：

```

/*
Copyright 2019 Google LLC

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    https://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
*/

import React from "react";
import { makeStyles } from "@material-ui/core/styles";
import Paper from "@material-ui/core/Paper";
import Typography from "@material-ui/core/Typography";
const useStyles = makeStyles(theme => ({
  root: {
    flexGrow: 1
  },
  paper: {
    width: "800px",
    margin: "0 auto",
    padding: theme.spacing(3, 2)
  }
}));
export default function Home() {
  const classes = useStyles();
  return (
    <div className={classes.root}>
      <Paper className={classes.paper}>
        <Typography variant="h5">
          Fancy Fashion & Style Online
        </Typography>
        <br />
        <Typography variant="body1">
          Tired of mainstream fashion ideas, popular trends and societal norms?
          This line of lifestyle products will help you catch up with the Fancy trend
          and express your personal style.
          Start shopping Fancy items now!
        </Typography>
      </Paper>
    </div>
  );
}

```

```
);  
}
```

您已更新 React 元件，但還得建構 React 應用程式，才能產生靜態檔案。執行下列指令即可建構 React 應用程式，並將其複製到單體公開目錄：

```
cd ~/monolith-to-microservices/react-app  
npm run build:monolith
```

程式碼更新完畢後，您需要重建 Docker 容器，並發布至 Container Registry。您可以使用與先前相同的指令，但這次要更新版本標籤！

執行下列指令，使用更新後的映像檔版本 2.0.0 觸發新的 Cloud Build：

```
cd ~/monolith-to-microservices/monolith  
  
#Feel free to test your application  
npm start  
  
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:2.0.0 .
```

在終端機視窗中按下 **Control+C** 鍵 (Windows 或 Mac)，停止網路伺服器程序。

在下一節中，您將使用該映像檔在不停機的情況下更新應用程式。

10. 更新網站，完全不必停機

變更完成，行銷團隊對您更新的內容非常滿意！現在是更新網站的好時機，而且不會中斷使用者的瀏覽體驗。請按照下列指示更新網站。

GKE 的滾動式更新機制可確保即使系統將舊容器映像檔的執行個體更換成新的容器映像檔，您的應用程式在所有運作中的備用資源之間依然保持運作且可用。

在指令列中，輸入下列指令，告知 Kubernetes 您要將 Deployment 的映像檔更新為新版本：

```
kubectl set image deployment/monolith
monolith=gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:2.0.0
```

輸出內容：

```
deployment.apps/monolith image updated
```

驗證部署作業

執行下列指令，即可驗證 Deployment 更新：

```
kubectl get pods
```

輸出內容：

NAME	READY	STATUS	RESTARTS	AGE
monolith-584fbc994b-4hj68	1/1	Terminating	0	60m
monolith-584fbc994b-fpwdw	1/1	Running	0	60m
monolith-584fbc994b-xsk8s	1/1	Terminating	0	60m
monolith-75f4cf58d5-24cq8	1/1	Running	0	3s
monolith-75f4cf58d5-rfj8r	1/1	Running	0	5s
monolith-75f4cf58d5-xm44v	0/1	ContainerCreating	0	1s

您會看到三個新 Pod 正在建立，舊 Pod 則關閉。從運作時間就能判斷哪些是新影片，哪些是舊影片。最後，您會再次看到三個 Pod，也就是三個更新後的 Pod。

如要確認變更，請再次前往負載平衡器的外部 IP，並注意應用程式已更新。

執行下列指令，列出服務並查看 IP 位址 (如果您忘記的話)：

```
kubectl get svc
```

您的網站應該會顯示您新增至首頁元件的文字！

Fancy Fashion & Style Online

Tired of mainstream fashion ideas, popular trends and societal norms? This line of lifestyle products will help you catch up with the Fancy trend and express your personal style. Start shopping Fancy items now!

11. 清除所用資源

刪除 Git 存放區

```
cd ~  
rm -rf monolith-to-microservices
```

刪除 Container Registry 映像檔

注意：如果您建立了其他版本，也可以使用相同語法刪除這些圖片。本程式碼研究室假設您只有兩個標記。

```
# Delete the container image for version 1.0.0 of our monolith  
gcloud container images delete gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:1.0.0 --quiet  
  
# Delete the container image for version 2.0.0 of our monolith  
gcloud container images delete gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:2.0.0 --quiet
```

從 Cloud Storage 刪除 Cloud Build 構件

注意：如果您使用 Cloud Build 處理這個程式碼研究室以外的構件，請務必手動從 Cloud Storage bucket

`gs://<PROJECT_ID>_cloudbuild/source` 刪除來源。

```
# The following command will take all source archives from all builds and delete them  
from cloud storage  
  
# Run this command to print all sources:  
# gcloud builds list | awk 'NR > 1 {print $4}'  
  
gcloud builds list | awk 'NR > 1 {print $4}' | while read line; do gsutil rm $line;  
done
```

刪除 GKE 服務

```
kubectl delete service monolith  
kubectl delete deployment monolith
```

刪除 GKE 叢集

```
gcloud container clusters delete fancy-cluster
```

注意：這個指令可能需要一點時間。

警告

如果收到未指定區域/可用區的錯誤訊息，請參閱環境設定一節，確認已設定預設運算可用區。

步驟 12 · 約 0 分鐘

12. 恭喜！

您已在 GKE 部署、調度資源及更新網站。您已有使用 Docker 和 Kubernetes 的經驗！

其他資源

- [Docker](#)
 - [Kubernetes](#)
 - [GKE](#)
 - [Cloud Build](#)
 - [Container Registry](#)
-